Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Реализация моделей безопасности в ОС Windows»

по дисциплине «Модели безопасности компьютерных систем»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Жуковский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания 3

2 Теоретические сведения 5

3 Результаты работы 12

4 Выводы 23

1 ФОРМУЛИРОВКА ЗАДАНИЯ

В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1 Изучить примеры драйверов, предоставленные разработчиками ОС Windows.

2 Изучить механизмы взаимодействия приложений уровня пользователя с драйверами (IOCTL).

3 Разработать формат хранения информации о объектах и субъектах системы и разрешенных правах доступа. Формат должен соответствовать варианту задания – дискреционной или мандатной политике доступа. Дискреционный контроль – для каждого объекта системы указывается перечень прав для субъектов, аналогично DACL. Мандатный контроль – объектам и субъектам назначаются метки (уровни доступа), аналогично Integirity Level/Mandatory Label.

4 Написать драйвер-фильтр, осуществляющий перехват операций доступа к указанным в варианте задания типам объектов и осуществляющий их блокировку, в случае, если они не разрешены в соответствии с указанными правилами. Конфигурационный файл с правилами должен храниться в соответствии с вариантом: в реестре или в файловой системе.

5 Написать приложение уровня пользователя, осуществляющее задание через консоль прав доступа, их сохранение в конфигурационный файл и инициирование обновления правил драйвером через механизм IOCTL-запросов.

6 Доступ к конфигурационному файлу/ключу реестра должен блокироваться для всех процессов, кроме приложения, устанавливающего правила доступа.

7 Изучить механизм нотификаторов. Добавить в разработанный драйвер возможность установки и снятия нотификатора на событие, указанное в варианте задания. Установка и удаление нотификатора должно происходить по команде, полученной через механизм IOCTL-запросов от разработанного приложения уровня пользователя. Установленный нотификатор должен

логировать в файл информацию о произошедшем событии, с указанием времени, именем и PID процесса, именем модуля.

8 Добавить в приложение уровня пользователя команды удаления и установки нотификаторов, отправляющие IOCTL-запросы драйверу.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Дискреционная модель доступа и матрица прав

Разграничение доступа отвечает на вопрос, какую операцию субъект может выполнить над объектом. Субъектом выступает процесс или учётная запись, объектом — файл, каталог или другой ресурс, а операция задаётся правом: прочитать, записать, создать, удалить. Пока правило не зафиксировано, программа, дошедшая до файла, распоряжается им в пределах учётной записи, от имени которой запущена. Модель выносит правило из кода программы и требует проверить его раньше, чем операция изменит данные. Обычная форма такой записи — матрица доступа: по одной оси стоят субъекты, по другой объекты, в клетке перечислены разрешённые права. Дискреционная модель заполняет матрицу по отдельным парам: у одного файла у разных субъектов разные клетки, и право, выданное одному субъекту, к остальным само не переходит. Мандатная модель устроена иначе: объектам и субъектам назначаются метки, метку назначает система, и доступ вычисляется сравнением меток, а не списком пар.

В Windows дискреционная проверка встроена в файловую систему. У файла NTFS есть дескриптор безопасности: владелец и список управления доступом. Список состоит из записей, каждая запись ссылается на идентификатор пользователя или группы и задаёт маску разрешённых прав. Проверка выполняется при открытии файла, а успешный результат закрепляется за описателем, поэтому повторные чтения и записи внутри уже открытого файла отдельной проверки не требуют. Важно различать два случая. Пустой список управления доступом не содержит ни одной разрешающей записи и запрещает доступ всем. Отсутствие списка, то есть объект без защиты, наоборот, не ограничивает никого. В работе эта разница воспроизведена на уровне текстовых правил: файл, которого нет в XML, доступен всем, а файл, который в XML есть, но не содержит строки для процесса, запрещает этому процессу доступ. Такое решение принято намеренно. Если бы файл, отсутствующий в политике, считался

запрещённым, фильтр закрыл бы системе доступ к собственным модулям, к журналу и к служебным файлам, и машина потеряла бы работоспособность.

В работе матрица упрощена до двух прав — чтения и записи — и до субъекта, которым является имя исполняемого файла. Это уже, чем учётная запись: права зависят не от пользователя, от имени которого запущен процесс, а от имени программы. Поэтому одна и та же программа, скопированная под другим именем, попадает под другие правила, и это хорошо видно на проверке с копией filetool.exe, переименованной в notepad.exe. Упрощение выбрано сознательно: оно делает модель наглядной и позволяет проверить её на одной тестовой программе. Настоящий список доступа NTFS при этом не подменяется: фильтр его не читает и не переписывает, а добавляет собственное решение по текстовой матрице.

2.2 Минифильтр файловой системы и высота

Прикладная программа к диску сама не обращается. Вызовы CreateFile, ReadFile и WriteFile переходят в интерфейс ядра, и диспетчер ввода-вывода оформляет каждый запрос пакетом IRP. Пакет проходит стек устройств тома и доходит до драйвера файловой системы; для тома NTFS это драйвер NTFS. Между программой и этим драйвером можно поставить фильтр. Прежний фильтр файловой системы встраивался в стек устройств напрямую и сам передавал запрос вниз, повторяя служебную работу файловой системы. Минифильтр устроен проще: он регистрируется у диспетчера фильтров FltMgr, сообщает ему свою высоту и список операций, а диспетчер сам выстраивает цепочку фильтров и вызывает зарегистрированные процедуры.

Высота задаёт порядок вызова. Фильтр с большей высотой видит запрос раньше фильтра с меньшей. Группа загрузки, в этой работе FSFilter Activity Monitor, относит драйвер к наблюдающим фильтрам, а место внутри группы определяется высотой. В работе выбрана высота 370030. На операцию можно поставить процедуру до выполнения и процедуру после выполнения. До

выполнения запрос ещё не обработан файловой системой: процедура либо пропускает его дальше, либо завершает сама и возвращает статус, в том числе STATUS_ACCESS_DENIED. После выполнения процедура видит уже полученный результат. Для модели доступа нужно первое, потому что запрет должен наступить до изменения объекта. Фильтр не заменяет драйвер тома и не выполняет его работу: он только наблюдает запросы и в нужный момент возвращает отказ.

2.3 Операции открытия, чтения и записи

Чтение и запись в матрице собираются из нескольких запросов ядра. Открытие файла — это IRP_MJ_CREATE: в нём передаются желаемый доступ и способ открытия, то есть открыть существующий файл, создать новый, перезаписать или заменить. Байты читает IRP_MJ_READ, байты пишет IRP_MJ_WRITE. Если программа открывает файл с перезаписью, прежняя длина обнуляется ещё при обработке открытия, и отдельный запрос записи для уничтожения содержимого уже не требуется. Именно так работает стандартный поток вывода файла: он открывает файл с созданием и усечением. Поэтому право «запись запрещена» проверяется не только на IRP_MJ_WRITE, но и на том открытии, которое само меняет файл. Если запретить только запись, файл успевает очиститься, хотя новые байты в него не попадают; запрет открытия не даёт усечению обойти правило.

Имя объекта в запросе обычно приходит как путь устройства тома, например \Device\HarddiskVolume3\каталог\файл. В конфигурации тот же файл записан буквой диска. Буква и номер тома обозначают путь по-разному, и перед сравнением с матрицей фильтр приводит имя к виду C:\каталог\файл, поскольку защищаемые файлы работы лежат на этом диске. Байты подкачки, помеченные как IRP_PAGING_IO, фильтр пропускает: они не связаны с прямым обращением программы к файлу. Относительный путь, открытый от дескриптора каталога, фильтр не разбирает и такой запрос пропускает; это ограничение модели, и в работе оно не используется.

2.4 Управляющее устройство и механизм IOCTL

Приложение уровня пользователя общается с драйвером через управляющее устройство. Драйвер создаёт устройство \Device\FsFilterX и символическую ссылку \DosDevices\FsFilterX, поэтому программа открывает его как \\.\FsFilterX обычным вызовом CreateFile. Дальше программа вызывает DeviceIoControl, а диспетчер ввода-вывода превращает вызов в IRP с главной функцией IRP_MJ_DEVICE_CONTROL и передаёт его обработчику драйвера. Код команды собирается макросом CTL_CODE из типа устройства, номера функции, метода передачи буфера и требуемого доступа. В работе используются тип FILE_DEVICE_UNKNOWN, метод METHOD_BUFFERED и доступ FILE_ANY_ACCESS, а номера функций — 0x800, 0x801 и 0x802. Команда 0x800 просит перечитать файл правил, 0x801 включает нотификатор, 0x802 выключает его.

Обработчик управляющего устройства проверяет, от какого процесса пришёл запрос, и принимает команды только от программы с именем fsctl.exe; остальным возвращается отказ. Перечитывание правил вынесено в отдельную команду не случайно. Разбор XML на каждой операции доступа был бы дорогим, а главное — запись в журнал нотификатора сама порождает файловые операции, и если бы правило перечитывалось на каждой из них, фильтр мог бы зацикливаться. Поэтому файл правил читается только при загрузке драйвера и по команде 0x800, а всё остальное время правила лежат в памяти и защищены блокировкой чтения-записи.

2.5 Защита файла правил

Файл правил защищён отдельно от общего списка объектов. Если бы он входил в политику на общих основаниях, то право записи в него, выданное одному процессу, позволило бы этому процессу добавить в XML разрешение для себя или для любой другой программы, и вся модель потеряла бы смысл. Поэтому драйвер проверяет путь C:\DriverConfig\policy.xml в операциях открытия, чтения и

записи особой ветвью: доступ разрешён только процессу с именем fsctl.exe и самому драйверу в тот момент, когда он перечитывает файл. Для собственного чтения драйвер выставляет флаг загрузки и запоминает процесс, который выполняет чтение; во всех остальных случаях запрос завершается отказом. Запись в XML, таким образом, не может выдать право другому процессу.

У этой защиты есть границы, и их нужно назвать прямо. Проверка идёт по имени исполняемого файла, поэтому копия любой программы, переименованная в fsctl.exe, сможет открыть файл правил: учётная запись и подпись программы здесь не проверяются. Удаление и переименование файла фильтр не перехватывает, потому что они приходят другими запросами, на которые процедуры не зарегистрированы. Наконец, защита действует только на пути, которые фильтр умеет разбирать; относительные открытия от дескриптора каталога он пропускает. Эти ограничения следуют из выбранной модели субъекта и не мешают проверке, но должны быть известны.

2.6 Нотификатор загрузки образа

Нотификатор — это механизм ядра, который вызывает зарегистрированную функцию при наступлении события. В варианте задания событием выбрана загрузка исполняемого модуля, поэтому используется PsSetLoadImageNotifyRoutine. Ядро вызывает колбэк при отображении образа в память и передаёт ему полное имя модуля, идентификатор процесса и сведения об образе. Колбэк выполняется в контексте процесса, который загружает модуль, на пассивном уровне, поэтому в нём допустимы обращения к файлам. Регистрация и снятие выполняются функциями PsSetLoadImageNotifyRoutine и PsRemoveLoadImageNotifyRoutine; повторная регистрация той же функции и снятие незарегистрированной не должны ронять драйвер, поэтому в коде стоят проверки состояния.

Журнал устроен так, чтобы не создавать рекурсию. Файл C:\DriverConfig\image.log открывается драйвером один раз, в обработчике команды включения нотификатора, и закрывается при выключении. В самом колбэке новый файл не открывается: строка дописывается в уже открытый описатель. Если бы колбэк открывал журнал заново, эта операция сама вызвала бы загрузку модулей и новый вызов колбэка. Дополнительно стоит флаг повторного входа, который не даёт писать журнал, если запись сама вызвала загрузку образа. Строка журнала содержит дату и время, идентификатор процесса, имя процесса и полный путь модуля. При выгрузке драйвера нотификатор снимается обязательно, а журнал закрывается, иначе колбэк остался бы в выгруженном коде.

2.7 Формат хранения правил

Формат правил выбран по варианту задания — XML. Файл имеет корневой элемент policy, внутри которого перечислены элементы file с атрибутом path, а у каждого file — элементы subject с тремя атрибутами: name, read и write. Значение 1 разрешает чтение или запись, значение 0 запрещает. Кодировка — UTF-8 без BOM, перевод строки — CRLF. Пространства имён, комментарии и подстановки XML-сущностей в формате не используются: разбор в ядре написан свой, построчный, без библиотеки XML, и лишние конструкции ему не нужны. Пределы, которые проверяет разбор: не больше 32 файлов, не больше 8 субъектов у файла, длина пути и имени процесса меньше 256 символов. Лишние записи не копируются в буфер.

Разбор устроен через промежуточный буфер. Драйвер читает файл целиком, разбирает его в отдельный массив, и только если разбор прошёл успешно, публикует новые правила под блокировкой. Поэтому ошибка в XML оставляет в памяти прежние правила, а команда перечитывания возвращает ошибку. Пустой или отсутствующий файл при первой загрузке означает пустую политику: обычные файлы разрешены, а файл правил всё равно закрыт для всех, кроме

fsctl.exe. Такое поведение выбрано, чтобы ошибка в конфигурации не блокировала систему.

3 РЕЗУЛЬТАТЫ РАБОТЫ

Работа выполнена на одной машине под Windows 11 сборки 28000. Для сборки установлены Visual Studio Community 2026, Windows Driver Kit и Windows SDK версии 10.0.28000, в состав Visual Studio добавлен набор инструментов WindowsKernelModeDriver10.0. Secure Boot на машине выключен. В конфигурации загрузки включён тестовый режим подписи, поэтому ядро принимает локальный тестовый сертификат. Драйвер подписан этим сертификатом по алгоритму SHA256, проверка signtool verify /pa завершается успешно; обход проверки подписи при старте системы не использовался.

Драйвер собран как минифильтр x64 Debug, программа управления — как консольное приложение x64 Release, программа-испытуемый filetool не менялась. Служба FsFilterX создана как драйвер файловой системы с зависимостью от FltMgr, группой загрузки FSFilter Activity Monitor и высотой 370030. Раздел установки в INF рассчитан на сборки до 25952, а установленная система имеет сборку 28000, поэтому фильтр загружается командой fltmc load, а не установщиком INF.

Для проверки используется консольная программа filetool: команда r с путём к файлу читает его и печатает строки, команда w с путём и текстом записывает текст. При отказе доступа программа печатает file openning failed; опечатка в этом сообщении сохранена как есть.

3.1 Формат правил

Файл правил лежит в C:\DriverConfig\policy.xml. В образце для файла note.txt заданы два субъекта: filetool.exe с правами read=1 и write=0 и notepad.exe с правами read=0 и write=0. Для файла report.txt задан filetool.exe с правами read=1 и write=1. Этого набора достаточно, чтобы проверить все три случая модели: разрешённое чтение, запрещённую запись и файл, которого нет в политике.

Разбор в ядре написан вручную и рассчитан ровно на этот вид файла. Драйвер читает файл в буфер, разбирает строки, проверяет атрибуты и пределы, складывает результат в промежуточный массив и публикует его под блокировкой только при успешном разборе. Ошибка разбора не меняет действующие правила. Файл C:\DriverConfig\Config.txt, оставшийся от предыдущей схемы, драйвер не читает и скрипт не удаляет: в нём помимо лабораторных путей есть строки для mspaint.exe.

3.2 Драйвер-фильтр

Драйвер FsFilterX зарегистрирован на операции IRP_MJ_CREATE, IRP_MJ_READ и IRP_MJ_WRITE. В предобработчике открытия он определяет желаемый доступ и способ открытия, в предобработчиках чтения и записи — путь файла и процесс, от имени которого пришёл запрос. Субъект определяется по образу процесса: драйвер запрашивает полное имя исполняемого файла и оставляет только имя без пути; если запрос полного имени невозможен, используется короткое имя из структуры процесса. Сравнение имён и путей выполняется без учёта регистра.

Путь файла приводится к виду с буквой диска C:. В открытии драйвер берёт имя из объекта файла и добавляет букву диска, в чтении и записи — запрашивает нормализованное имя у диспетчера фильтров и заменяет путь устройства тома на букву диска. Каталоги и байты подкачки пропускаются. Если файла нет в политике, доступ разрешён. Если файл есть, но процесса в его списке нет, доступ запрещён. Если процесс есть, смотрится его флаг: единица разрешает операцию, ноль запрещает. Запрет возвращается как STATUS_ACCESS_DENIED и завершает запрос до того, как файловая система его выполнит.

Для файла правил действует отдельная ветвь, описанная в теоретической части: открывать его может только fsctl.exe и сам драйвер при перечитывании. Журнал нотификатора, наоборот, не защищается: его дописывает драйвер, а

читать его можно, пока нотификатор включён. Перечитывание правил вызывается только из загрузки драйвера и из обработчика команды 0x800; в предобработчиках файловых операций вызова перечитывания нет.

3.3 Управляющее устройство и IOCTL

Управляющее устройство создаётся при загрузке драйвера вместе с символической ссылкой. Драйвер обрабатывает открытие, закрытие, очистку и управляющий запрос только для этого устройства; запросы томов по-прежнему идут в минифильтр. Коды команд вынесены в общий заголовочный файл, который подключают и драйвер, и программа управления, чтобы номера не разъезжались. Обработчик проверяет имя процесса и принимает команды только от fsctl.exe. При выгрузке драйвер снимает нотификатор, закрывает журнал, удаляет символическую ссылку и устройство и только после этого снимает фильтр.

3.4 Программа fsctl

Программа fsctl — консольное приложение с командами set, show, reload и notify on либо notify off. Команда set принимает путь, имя процесса и два флага, читает текущий XML, меняет одну пару «файл и процесс», сохраняя остальные строки, записывает файл и отправляет команду перечитывания. Повторный set для той же пары заменяет флаги, а не добавляет вторую строку. Команда show печатает текущий XML и драйвер не трогает. Команда reload только отправляет команду перечитывания. Команды notify on и notify off только ставят и снимают нотификатор.

Порядок действий в set выбран так, чтобы диск и ядро не разошлись. Сначала программа проверяет, что устройство драйвера открывается: если фильтр не загружен, программа сообщает об этом и файл не меняет. Затем она читает XML, меняет нужную пару, записывает файл и отправляет команду перечитывания. Если команда не удалась, программа возвращает на диск прежний

XML и просит драйвер перечитать его ещё раз. Поэтому неудачное изменение не оставляет на диске правила, которых нет в ядре.

3.5 Нотификатор и журнал

Нотификатор ставится командой notify on: драйвер открывает журнал C:\DriverConfig\image.log на дописывание и регистрирует колбэк загрузки образа. Команда notify off снимает колбэк, дожидается уже вошедшего вызова и закрывает журнал. Пока нотификатор включён, каждая загрузка модуля дописывает строку с датой и временем, идентификатором процесса, именем процесса и полным путём модуля. При выгрузке драйвера нотификатор снимается автоматически, поэтому колбэк не остаётся в выгруженном коде, а состояние нотификатора после выгрузки не сохраняется: новая загрузка начинает с выключенного журнала.

3.6 Сборка, подпись и установка

Сборка драйвера выполняется командой msbuild FsFilterX\FsFilterX.sln с параметрами Configuration=Debug и Platform=x64, сборка программы управления — командой msbuild fsctl\fsctl.vcxproj с параметрами Configuration=Release и Platform=x64. Готовый драйвер лежит в FsFilterX\FsFilterX\x64\Debug\FsFilterX.sys, готовая программа — в fsctl\x64\Release\fsctl.exe. Подпись выполняется signtool по алгоритму SHA256 сертификатом WDKTestCert winuser из хранилища текущего пользователя, отпечаток сертификата 906AA1ADCD98DCBC641051CCBEFA9BAADFAB2206. Подписанный файл копируется в C:\Windows\System32\drivers\FsFilterX.sys, после чего фильтр загружается командой fltmc load FsFilterX и проверяется командой fltmc filters.

3.7 Скрипт проверки

Для повторения прогона написан скрипт install-filter.ps1, запускаемый от администратора. Скрипт проверяет права администратора и тестовый режим подписи, подписывает драйвер, выгружает ранее загруженный фильтр, заново

кладёт файл правил и тестовые файлы, копирует драйвер в системный каталог, проверяет подпись, создаёт службу и параметры экземпляра, загружает фильтр и прогоняет проверки. Результат каждой проверки и весь вывод команд записываются в файл install-result.txt. Последний полный прогон завершился со строкой FAILED 0, то есть все проверки прошли. Файл C:\DriverConfig\Config.txt скрипт не удаляет и не перезаписывает.

3.8 Результаты работы

Полный прогон скрипта install-filter.ps1 завершился успешно: в журнале прогона строка FAILED 0. Ниже приведены снимки по каждому пункту проверки. Команды показаны целиком, вместе с путями и аргументами.

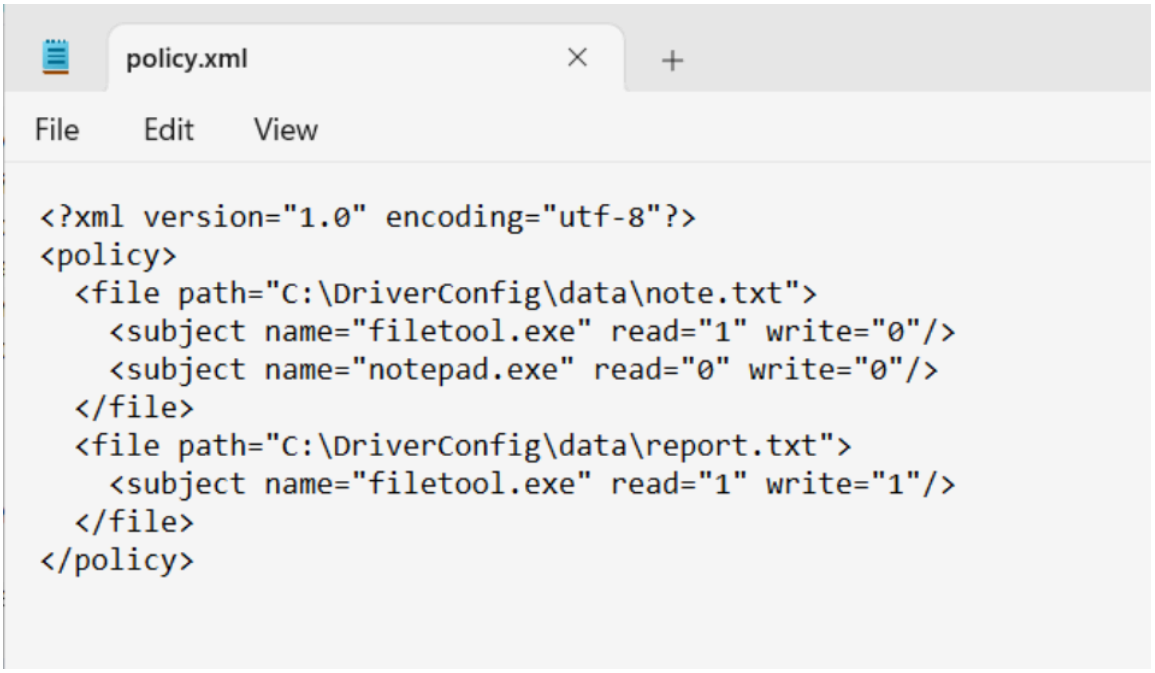


Рисунок 1 – Файл правил policy.xml

Перед загрузкой фильтра файл C:\DriverConfig\policy.xml открыт в блокноте. В нём видно объявление XML, корневой элемент policy и две записи file: для note.txt с субъектами filetool.exe с правами read=1 и write=0 и notepad.exe с правами read=0 и write=0, а также для report.txt с субъектом filetool.exe с правами read=1 и write=1. Этот файл и стал основой политики, которую драйвер прочитал при загрузке.



Рисунок 2 – Тестовый режим подписи драйверов и сборка 28000

На снимке рабочий стол с надписью Test Mode в правом нижнем углу и окно winver, которое показывает Windows 11 сборки 28000. Тестовая подпись включена: команда bcdedit /enum {current} в журнале прогона выводит testsigning Yes. Без этого режима ядро не загрузило бы драйвер, подписанный локальным тестовым сертификатом.

```
C:\Windows\System32>fltmc load FsFilterX

C:\Windows\System32>fltmc filters

Filter Name                     Num Instances    Altitude    Frame
------------------------------  -------------  ------------  -----
bindflt                                 1       409800         0
UCPD                                    8       385250.5       0
FsFilterX                               8       370030         0
WdFilter                                8       328010         0
applockerfltr                           7       265000         0
storqosflt                              0       244000         0
```

Рисунок 3 – Загруженный минифильтр FsFilterX

Команда fltmc filters показывает в списке фильтров строку FsFilterX с высотой 370030 и подключёнными экземплярами тома. Фильтр стоит в группе FSFilter Activity Monitor и виден диспетчеру фильтров наряду со штатными фильтрами системы.

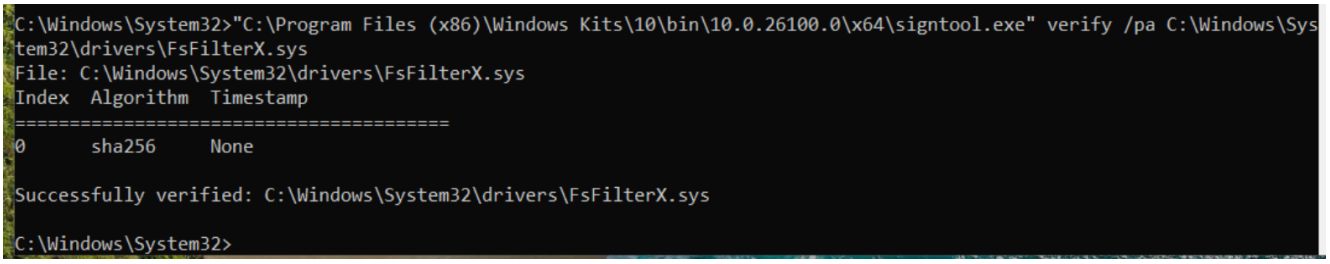


Рисунок 4 – Проверка тестовой подписи драйвера

Команда signtool verify /pa для файла C:\Windows\System32\drivers\FsFilterX.sys показывает алгоритм sha256 и строку Successfully verified. Подпись сделана тестовым сертификатом WDKTestCert winuser, отпечаток 906AA1ADCD98DCBC641051CCBEFA9BAADFAB2206; проверка политики подписи проходит.

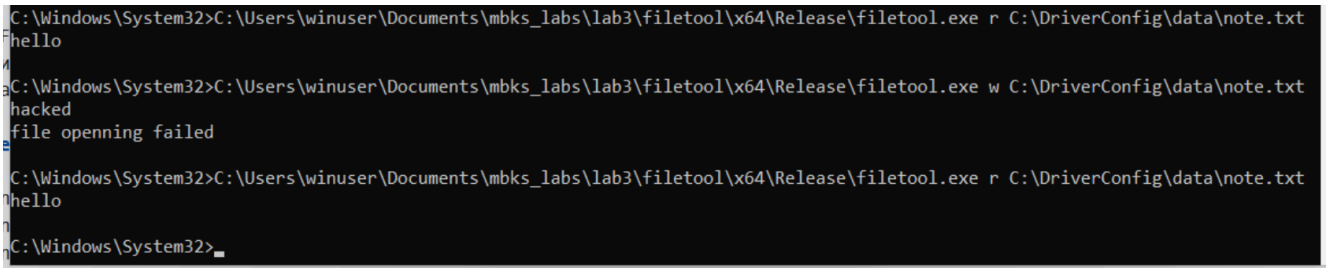


Рисунок 5 – Чтение note.txt разрешено, запись запрещена

Программа filetool читает C:\DriverConfig\data\note.txt и получает сохранённый текст hello. Запись строки hacked в тот же файл отклоняется: программа печатает file openning failed. Повторное чтение снова показывает hello, то есть запрещённое открытие файл не изменило и старое содержимое не стёрлось.

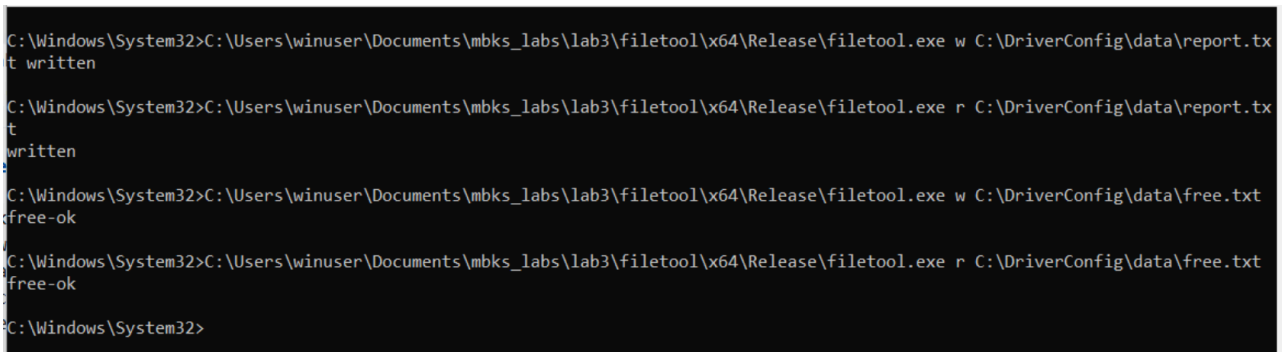


Рисунок 6 – Запись report.txt разрешена, файл вне политики не ограничивается

Запись и чтение C:\DriverConfig\data\report.txt проходят: в файле оказывается текст written. Файл C:\DriverConfig\data\free.txt в политике не упомянут, поэтому он доступен любой программе: запись free-ok и последующее чтение проходят. Этим проверены два разных случая — разрешённый субъект для файла из политики и отсутствие ограничений для файла, которого в политике нет.

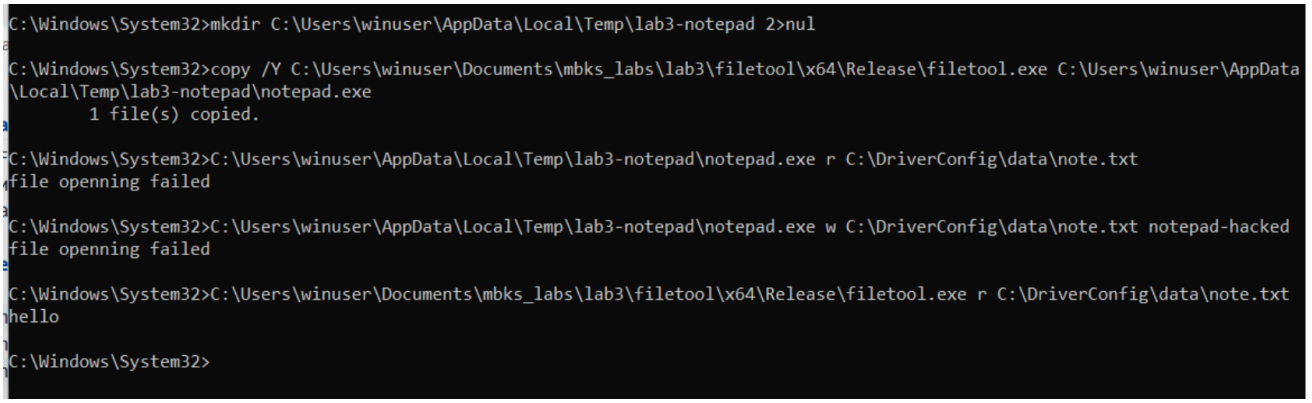


Рисунок 7 – Копия filetool.exe под именем notepad.exe не допущена к note.txt

Копия filetool.exe, запущенная как notepad.exe, не может ни прочитать, ни записать note.txt: оба вызова печатают file openning failed. Последующее чтение тем же filetool.exe по-прежнему показывает hello. Значит, решение принимается по имени исполняемого файла, а не по самой программе, и запрещённое открытие не изменило файл.

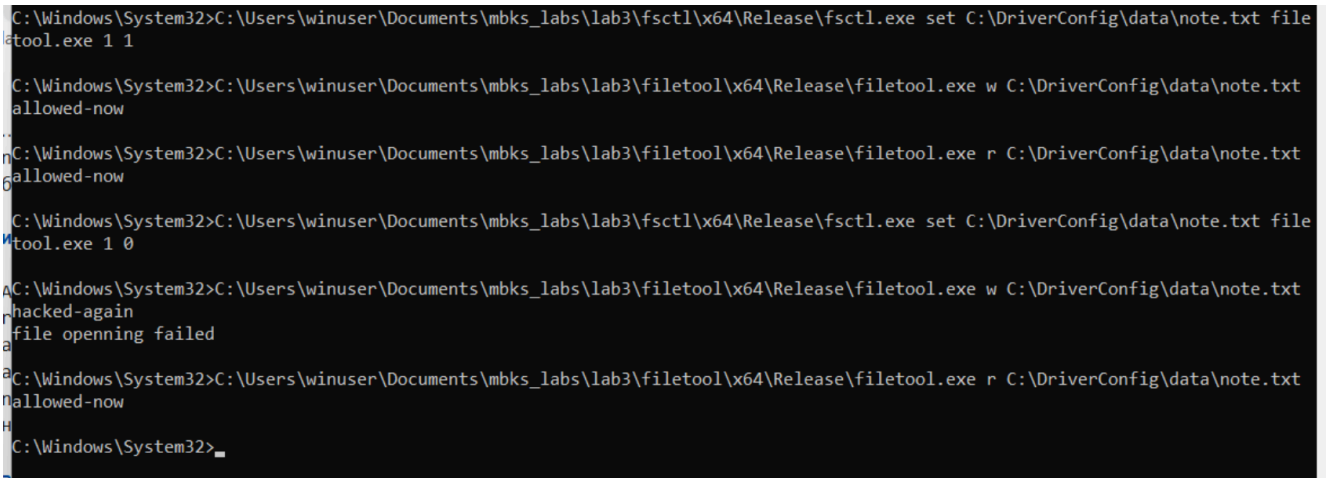


Рисунок 8 – Смена права записи через fsctl set без перезагрузки

Команда fsctl set C:\DriverConfig\data\note.txt filetool.exe 1 1 печатает «правила обновлены», после чего запись строки allowed-now в note.txt проходит: программа ничего не печатает, а чтение показывает allowed-now. Затем команда fsctl set с флагами 1 и 0 возвращает запрет: запись hacked-again снова получает file openning failed, а в файле остаётся allowed-now. Перезагрузка и повторная загрузка фильтра не выполнялись: правило изменилось в памяти драйвера по команде перечитывания.

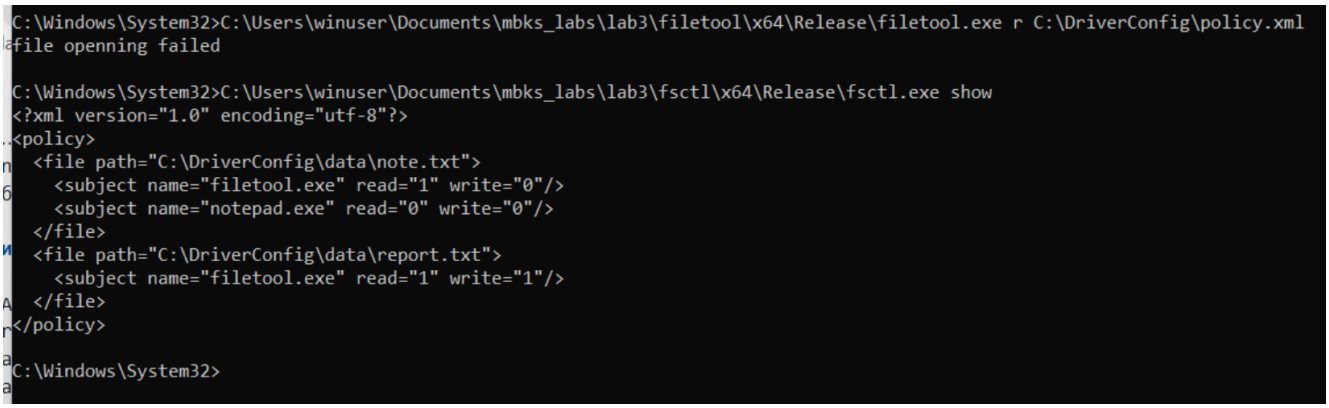


Рисунок 9 – Файл правил защищён от посторонних процессов

Программа filetool не может прочитать C:\DriverConfig\policy.xml: вызов печатает file openning failed. Программа fsctl командой show тот же файл читает и печатает его содержимое. Значит, защита файла правил действует отдельно от

общего списка объектов и пропускает только программу управления и сам драйвер.

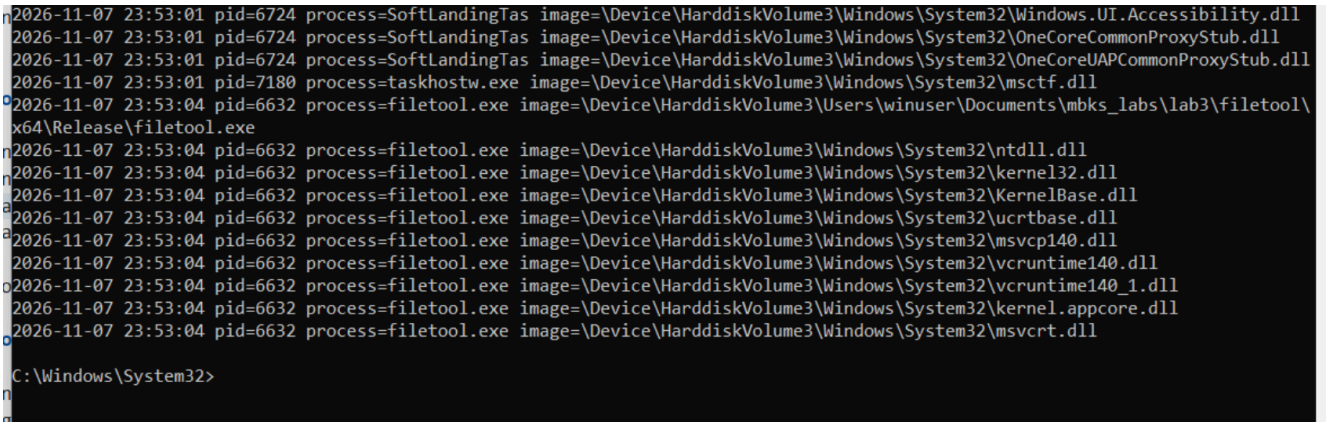


Рисунок 10 – Нотификатор загрузки образов и журнал image.log

Команда fsctl notify on включает нотификатор, после чего запуск filetool добавляет в C:\DriverConfig\image.log строки с датой и временем, идентификатором процесса, именем процесса и полным путём модуля. В журнале есть строки, где process=filetool.exe: каждая такая строка начинается с даты и времени, затем идут pid, имя процесса и полный путь загруженного модуля. Время, PID, имя процесса и имя модуля присутствуют, как и требует задание.

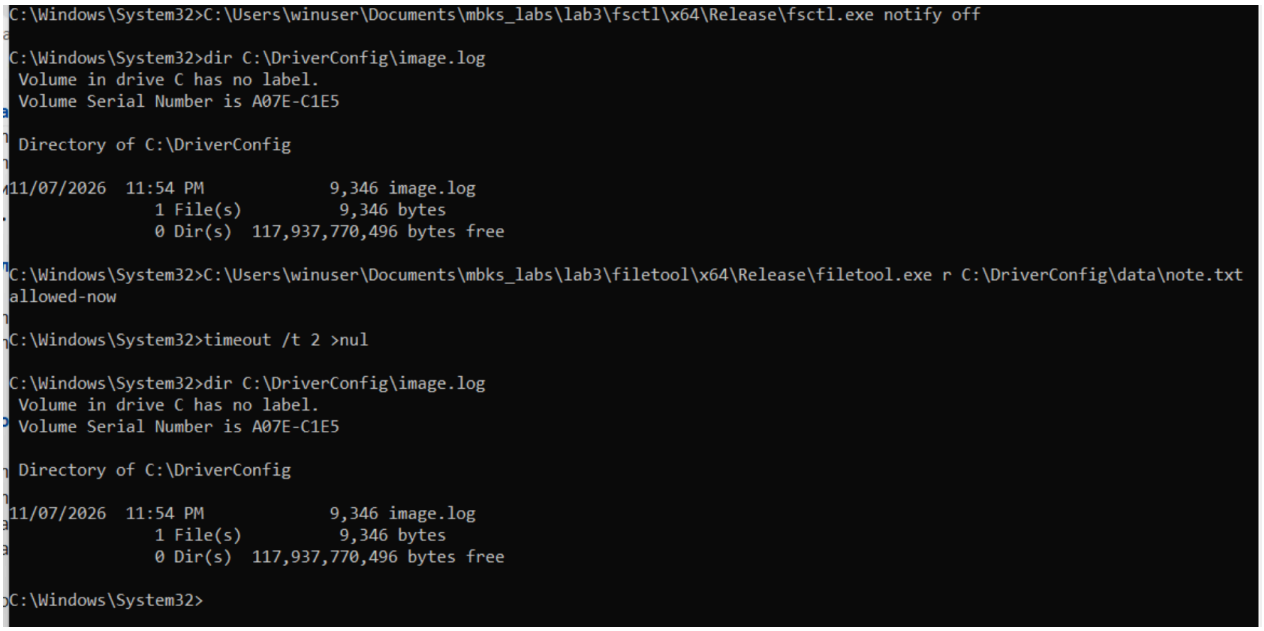


Рисунок 11 – Нотификатор снят, новые строки в журнал не пишутся

Команда fsctl notify off снимает нотификатор и закрывает журнал. Размер файла C:\DriverConfig\image.log, снятый командой dir до и после запуска filetool, одинаковый: новый запуск программы новых строк не добавил. Само чтение note.txt при этом по-прежнему проходит и показывает allowed-now. Значит, снятие нотификатора закрывает журнал и не мешает файловым операциям.

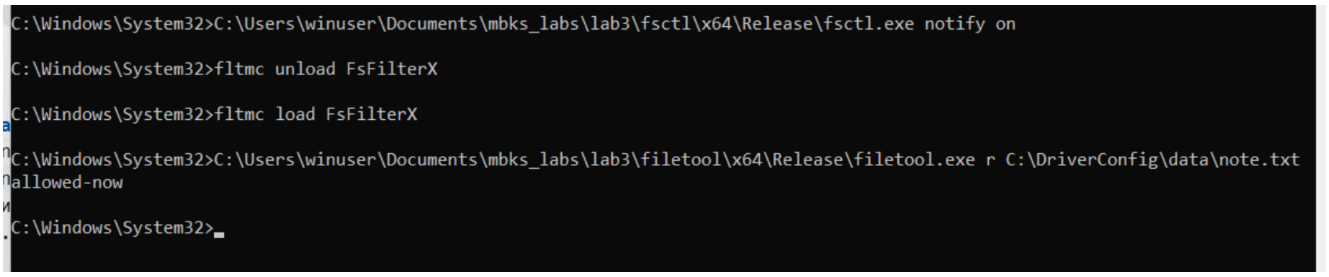


Рисунок 12 – Выгрузка фильтра при включённом нотификаторе и повторная загрузка

Нотификатор включён командой fsctl notify on, после чего фильтр выгружается командой fltmc unload FsFilterX и снова загружается командой fltmc load FsFilterX. Обе команды проходят без ошибок. После загрузки filetool снова читает note.txt и получает allowed-now: драйвер заново прочитал файл правил и восстановил политику. Выгрузка не оставила колбэк в выгруженном коде и не привела к падению системы.

4 ВЫВОДЫ

В работе на уровне ядра Windows реализован дискреционный контроль доступа к файлам. Права хранятся в XML-файле: у каждого объекта перечислены субъекты с двумя признаками, чтением и записью. Минифильтр применяет эти правила к операциям открытия, чтения и записи и отклоняет запрещённую операцию, не уничтожая уже записанные данные. Проверка на filetool.exe и на копии с именем notepad.exe показывает, что решение зависит от пары «процесс — файл», а файл, не внесённый в политику, остаётся доступным.

Добавлены управляющее устройство и механизм IOCTL: программа fsctl задаёт права из консоли, сохраняет их в файл и просит драйвер перечитать правила без перезагрузки. Файл правил защищён отдельно: его открывает только программа управления и сам драйвер при перечитывании. Нотификатор загрузки образов ставится и снимается по команде и пишет в журнал время, идентификатор и имя процесса и полный путь модуля; выгрузка драйвера при включённом нотификаторе проходит, а после новой загрузки служба снова работает.

У работы есть границы, которые следуют из выбранной модели. Субъект — имя исполняемого файла, а не учётная запись Windows, поэтому копия программы, переименованная в fsctl.exe, сможет открыть файл правил. Файлы, которых нет в XML, не ограничиваются. Удаление и переименование фильтр не перехватывает, а пути и имена в правилах рассчитаны на латиницу и диск C:. Журнал нотификатора в политике не защищается. В этих рамках цель работы достигнута: фильтр перехватывает и блокирует запрещённые операции доступа к файлам, правила обновляются по IOCTL, файл правил защищён, нотификатор загрузки образов работает и ведёт журнал.